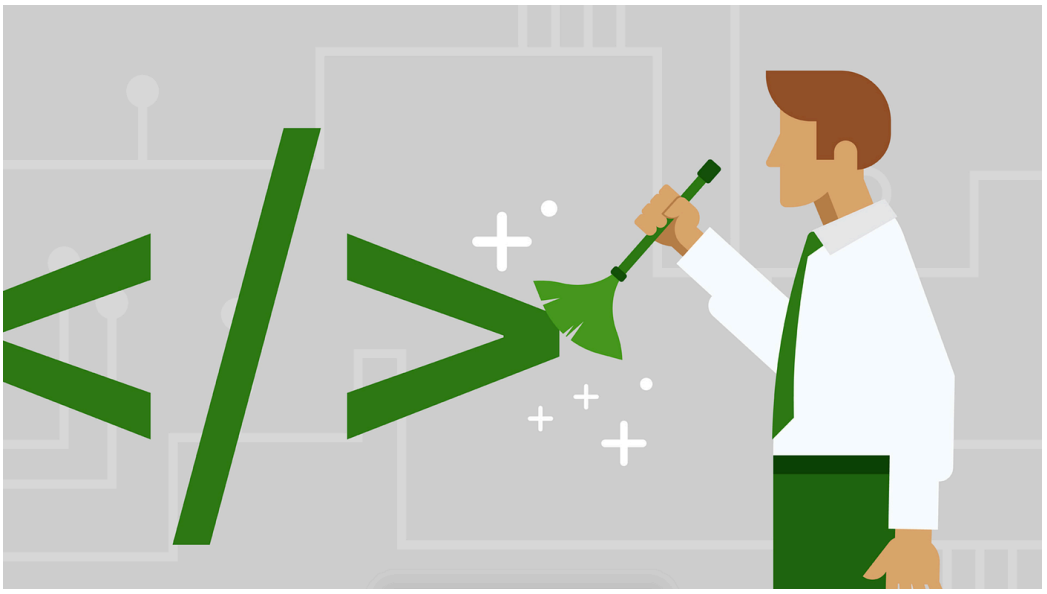
Desarrollo de Software

Análisis de publicaciones relacionadas con el Refactoring



Monica Erce Muñoz - 100550243

Lucía Ramírez Sánchez - 100550254

Cloe Paz Morales - 100550784

G 84

ÍNDICE

Artículo 1 3
Artículo 2 3
Artículo 3 4
Artículo 5 5
Artículo 6 5

Artículo 1

- **Título**: *La Inteligencia Artificial como Herramienta para el Desarrollo de Código Limpio y de Alta Calidad.*
- **Publicación**: orcid.org
- **Dirección Web:** https://www.researchgate.net/profile/Jose-Mamani-Figueroa-2/publication/398681626_La_Inteligencia_Artificial_como_Herramienta_para_el_Desarrollo_de_Codigo_Limpio_y_de_Alta_Calidad/links/693fef100c98040d481df522/La-Inteligencia-Artificial-como-Herramienta-para-el-Desarrollo-de-Codigo-Limpio-y-de-Alta-Calidad.pdf
- **Autor:** José David Mamani Figueroa
- **Fecha de publicación**: Diciembre 2025

El texto empieza hablando sobre la importancia de tener un código legible y fácil de comprender y mantener en el tiempo, y cómo la inteligencia artificial se relaciona con esto. El uso de la inteligencia artificial nos genera cierta dependencia (cada vez más) a la hora de generar un buen código, y esto nos lleva a la pérdida de comprensión del código por parte de los desarrolladores y surgen dudas éticas relacionadas con la propiedad intelectual del código y el uso de la IA de grandes repositorios.

Se busca orientar a reducir el uso de la IA y proporcionar un enfoque educativo utilizando sus beneficios. Esta metodología se basa en identificar patrones, tendencias y limitaciones del uso de la IA en el desarrollo de código de software limpio, para garantizar la calidad del código.

Los resultados de esta metodología evidencian que la IA desempeña un papel importante en el refactoring y en obtener un código limpio; se obtiene mejor calidad y legibilidad del código, un incremento en la productividad del programador o detección temprana de errores y malas prácticas de programación.

Concluyendo que la IA facilita el desarrollo del refactoring, siempre y cuando se tenga un criterio técnico adecuado para evitar errores conceptuales o dependencias innecesarias.

Artículo 2

- **Título**: *REFACTORIZACIÓN Y DESARROLLO DE PRUEBAS DE VERIFICACIÓN DEL CONTROLADOR DE REDES DEFINIDAS POR SOFTWARE (SDN) PERIPLUS.*
- **Publicación**: burjcdigital.urjc.es
- **Dirección Web**: https://burjcdigital.urjc.es/items/5f06a9ba-1c11-42e2-ba71-d039527dd89b
- **Autor**: Elena Ballesteros Atienza
- **Fecha de publicación**: 25 junio 2024

Este documento comienza resaltando la importancia de hacer un plan de pruebas de software (por ejemplo unitarias) antes de empezar a mejorar o limpiar el código. Tras obtener estas pruebas comienza el proceso de refactorización, donde el objetivo no es añadir nuevas funcionalidades al código, si no mejorar la legibilidad, reducir la complejidad, eliminar el código duplicado y hacer que sea más fácil de mantener a futuro y entendible.

Finalmente, tras mejorar el código se vuelven a ejecutar todas las pruebas del primer paso y verificar si dan buenos resultados.

Artículo 3

- **Título:** *Agentic Refactoring An Empirical Study of AI Coding Agents*
- **Revista o congreso dónde ha sido publicado:** En arXiv (Prepint, enviado a *ACM Transactions on Software Engineering and Methodology - TOSEM*)
- **Dirección web:** https://arxiv.org/abs/2511.04824
- **Autores:** Kosei Horikawa, Hao Li, Yutaro Kashiwa, Bram Adams, Hajimu Iida y Ahmed E. Hassan.
- **Fecha de publicación:** 6 de noviembre de 2025

Este artículo investiga cómo los agentes de inteligencia artificial asumen de forma autónoma tareas de refactorización en proyectos de código abierto. Tras analizar miles de intervenciones, los autores descubrieron que la IA dedica gran parte de su esfuerzo (aproximadamente el 26% de sus *commits*) a refactorizar. Sin embargo, el estudio afirma que estos agentes son muy eficientes realizando cambios estructurales de bajo nivel para mejorar la consistencia, la legibilidad y reducir la complejidad (como el renombramiento de variables o ajustes de tipos), pero aún tienen dificultades con refactorizaciones arquitectónicas complejas que necesitan una visión global del sistema. Este trabajo resulta de gran relevancia para nuestra investigación porque documenta con datos empíricos el cambio de paradigma desde el uso de herramientas manuales tradicionales hacia agentes autónomos de codificación, definiendo el potencial y las limitaciones actuales de la IA en el mantenimiento del software.

Artículo 4

- **Título:** Refactoring with LLMs: Bridging Human Expertise and Machine Understanding
- **Revista o congreso dónde ha sido publicado::** arXiv (Preprint)
- **Dirección web:** https://arxiv.org/abs/2510.03914
- **Autores:** Yonnel Chen Kuang Piao, Jean Carlors Paul, Leuson Da Silva, Arghavan Moradi Dakhel, Mohammad Hamdaqa y Foutse Khomh.
- **Fecha de publicación:** 4 de octubre de 2025

Este artículo explora cómo las indicaciones creadas por expertos humanos pueden utilizarse para mejorar la capacidad de los Grandes Modelos de Lenguaje (LLMs) a la hora de automatizar tareas de *refactoring*. Los autores tradujeron el catálogo de refactorización de Martin Fowler (con sus motivaciones, pasos y objetivos) en estrategias de instrucciones para guiar a inteligencias artificiales modernas, logrando que estas puedan ejecutar 61 tipos distintos de refactorización manteniendo la semántica y el funcionamiento del programa en entornos reales. El estudio es relevante porque propone integrar los conocimientos humanos de forma estructurada en los agentes de codificación para conseguir un mantenimiento de software automatizado pero de alta calidad.

Artículo 5

- **Título:** *Propuesta de arquitectura de control de versiones en bases de datos evolutivas.*
- **Publicación:** Documento final de Proyecto de Investigación Aplicada 2, Maestría en Ingeniería de Software con énfasis en Arquitectura y Diseño de Software, Universidad CENFOTEC.
- **Dirección web:** https://repositorio.ucenfotec.ac.cr/bitstream/handle/123456789/623/PIA-SALAS%20MONTOYA%20GERARDO-MISAD-mar2025%20%281%29.pdf?sequence=1&isAllowed=y
- **Autores:** Gerardo Adolfo Salas Montoya y Miguel Pérez Montero.
- **Fecha de publicación:** marzo de 2025.

Este artículo trata sobre cómo mejorar la forma en que se actualizan las bases de datos cuando un software está en desarrollo constante. El autor explica que muchas veces las bases de datos se quedan atrás o dan errores porque se gestionan a mano. Por eso, propone usar herramientas como Liquibase o Flyway para que los cambios en las tablas y datos sean automáticos y queden registrados, igual que se hace con el código normal. Básicamente, busca que los despliegues sean más seguros y que no se pierda el control de qué versión de la base de datos estamos usando.

Me parece importante porque casi siempre nos centramos en el código de la app y nos olvidamos de la base de datos. Este texto da ideas prácticas para que no haya fallos cuando el equipo sube cambios a producción, haciendo que todo el proceso de trabajo sea más ordenado.

Artículo 6

- **Título:** *Patrones de diseño y estrategias reutilizables en la automatización de pruebas de software con Selenium y Java: Page Object Model, Singleton, Factory y Fluent Interface.*
- **Publicación:** Revista Politécnica de Aguascalientes.
- **Dirección web:** https://revistapolitecnicaags.upa.edu.mx/wp-content/uploads/2025/11/V5109.pdf
- **Autores:** Sánchez, Gilberto.
- **Fecha de publicación:** 4 de mayo de 2025.

El artículo habla de cómo usar trucos de programación (llamados patrones de diseño) para que las pruebas que se hacen automáticamente con Selenium no sean un desastre. El autor hizo un experimento comparando scripts de prueba normales contra otros bien organizados con patrones cómo "Page Object Model" o "Singleton". Los resultados fueron muy buenos: se escribió mucho menos código repetido y las pruebas corrían más rápido. Al final, demuestra que si organizas bien el código de las pruebas desde el principio, es mucho más fácil arreglar fallos después.

Lo elegí porque a veces las pruebas automáticas fallan mucho o son difíciles de mantener cuando la web cambia un poco. Este artículo explica de forma clara cómo hacer que esas pruebas sean más robustas y profesionales, ahorrando tiempo y trabajo al equipo de calidad.